KLAi

Gradient descent: training is walking downhill on the loss, one step at a time

How does a model with millions or billions of numbers find good values for all of them?

Mathematics for AI and Machine Learning | Session 10 of 18 | Calculus and optimisation

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Apply the update rule w = w - eta times the gradient by hand, for one step and for several steps



Explain what happens when the learning rate is too small or too large, and what mini-batches, momentum and Adam change



Write the gradient descent loop for a line fit in Python, or one row per step in a spreadsheet, and compare four learning rates

Your two hours: learn the update rule, then watch it run

0:00	10	Recall and a real question from AI
0:10	35	Concept: a picture, then a hand example
0:45	10	Break
0:55	45	Lab: Python or spreadsheet, predict, run, change
1:40	15	Where it appears in AI, plus an exit check
1:55	5	Save the work

Pictures and hand calculations first. Then the lab: a loop in Python or one row per step.

Three questions from last time

1. Use the chain rule: what is the derivative of y = (3x plus 1) squared with respect to x?

2. A function takes 3 inputs and gives 2 outputs. What shape is its Jacobian matrix?

3. Why does backpropagation need the chain rule?

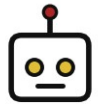
Answer aloud first. Then we check together.

Answers to the three questions from Session 9

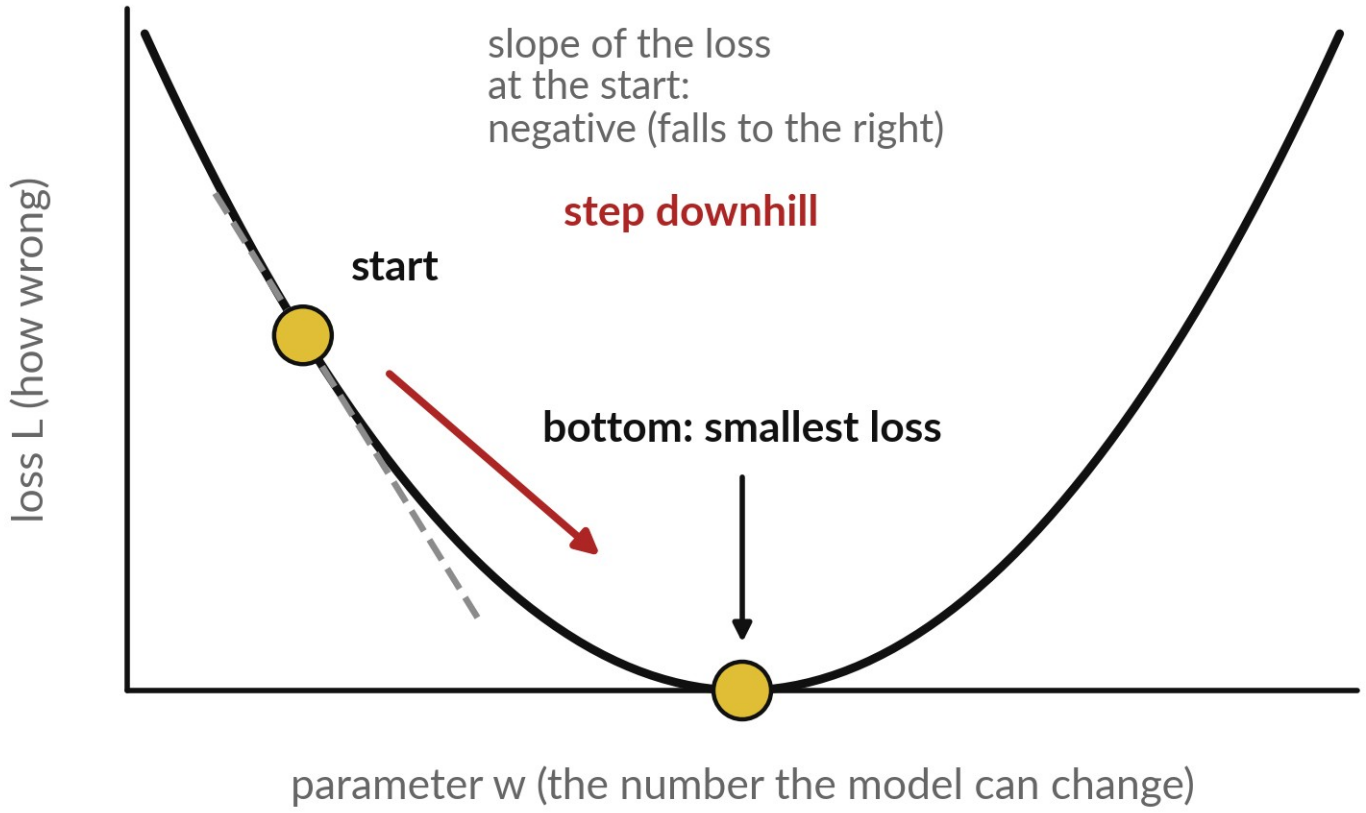
Question	Short answer
Use the chain rule: what is the derivative of y = (3x plus 1) squared with respect to x?	It is 6(3x + 1): the outer derivative 2(3x + 1) multiplied by the inner derivative 3.
A function takes 3 inputs and gives 2 outputs. What shape is its Jacobian matrix?	Two rows and three columns: one row for each output and one column for each input.
Why does backpropagation need the chain rule?	A network is a function of functions, so the slope of the loss for an early weight is a product of the slopes along the path to the loss.

Check your answers. If one surprised you, look at Session 9 before the lab.

Words we use today

	Word	What it means
	**Gradient descent**	Repeatedly changing the parameters a little in the direction that lowers the loss.
	**Learning rate (eta)**	The size of each step. We write it as the Greek letter eta; some books write alpha.
	**Epoch and mini-batch**	An epoch is one pass through all the training rows. A mini-batch is a small group of rows used for one step.
	**Momentum**	A running memory of past gradients that smooths the path and speeds it up.
	**Adam**	An optimiser (Kingma and Ba, 2014) that gives each parameter its own step size.

Training a model means walking downhill on the loss



The loss L measures how wrong the model is. Training lowers it.

The curve shows the loss for each value of one parameter w.

At the start the slope is negative, so the loss falls if w grows. We step that way.

The bottom of the curve is the best value of w.

The update rule: new w equals old w minus the learning rate times the gradient

One parameter

$$w \leftarrow w - \eta \frac{dL}{dw}$$

Many parameters

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$$

Symbol	What it means	In our data
w	The parameter we improve. The arrow means: replace the old value. Bold w is the column of all the parameters (Session 8 also calls it theta).	start at w = 0
eta (learning rate)	The size of each step. Some books write alpha.	eta = 0.1
dL/dw	The slope of the loss with respect to w (a derivative).	-6 at w = 0, for L = (w - 3)^2
nabla L	The gradient: the column of all partial derivatives (Session 8).	one entry per parameter

Worked: w = 0 - 0.1 × (-6) = 0.6. The minus sign moves w the way that lowers the loss.

Five steps by hand on the bowl (w minus 3) squared with eta = 0.1: the steps shrink by themselves

Step	w before	Slope 2(w minus 3)	w after = w before minus 0.1 × slope
1	0	-6	0.6
2	0.6	-4.8	1.08
3	1.08	-3.84	1.464
4	1.464	-3.072	1.771
5	1.771	-2.458	2.017

Each step closes one fifth of the remaining gap to 3. The best value is w = 3, where the slope is 0.

A line fit becomes a loss: a straight-line model and its mean squared error

Model

$$\hat{y}_i = b + w\,x_i$$

Loss (MSE)

$$L(w, b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Symbol	What it means	In our data
x_i	Centred distance of row i: distance minus the average 4.5 km.	x = -3.5, -2.5, ..., 3.5
y_i, y-hat_i	Actual fare and predicted fare of row i, in thousand UGX.	illustrative fares 3.2 to 10.3
b, w	Intercept (the fare at the average distance) and slope (thousand UGX per km).	both start at 0
n	The number of rows.	n = 8 trips

Start: the loss is the mean of y squared, 52.11. Centred x adds up to 0, so ∂L/∂w does not involve b.

The gradient of the line-fit loss has one entry for w and one for b

Gradient

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\,x_i \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

Symbol	What it means	In our data
∂L/∂w	How fast the loss changes when w changes (a partial derivative, Session 8).	-11.025 at w = 0, b = 0
∂L/∂b	How fast the loss changes when b changes.	-13.600 at w = 0, b = 0
(y-hat_i minus y_i)	The error of row i. Positive means too high.	at the start: -y_i
2/n and the sums	Add over the rows, then multiply by 2/n.	2/8 = 0.25; sum of error × x = -44.10; sum of errors = -54.4

Both slopes are negative here, so the update raises both w and b.

Step 1 of the line fit by hand, starting from w = 0 and b = 0

Row i	x (km)	y (thousand UGX)	Error: y-hat minus y	Error times x
1	-3.5	3.2	-3.2	11.20
2	-2.5	4.1	-4.1	10.25
3	-1.5	5.4	-5.4	8.10
4	-0.5	5.9	-5.9	2.95
5	+0.5	7.6	-7.6	-3.80
6	+1.5	8.0	-8.0	-12.00
7	+2.5	9.9	-9.9	-24.75
8	+3.5	10.3	-10.3	-36.05

Sums: error times x = -44.10, error = -54.4. $\partial L/\partial w = 0.25 \times (-44.10) = -11.025$, $\partial L/\partial b = 0.25 \times (-54.4) = -13.600$. With eta = 0.1: w = 1.1025, b = 1.3600.

Mean, half-sum and full-sum squared error share one minimum; only the step size changes

MSE (today)

$$L_{MSE} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$$

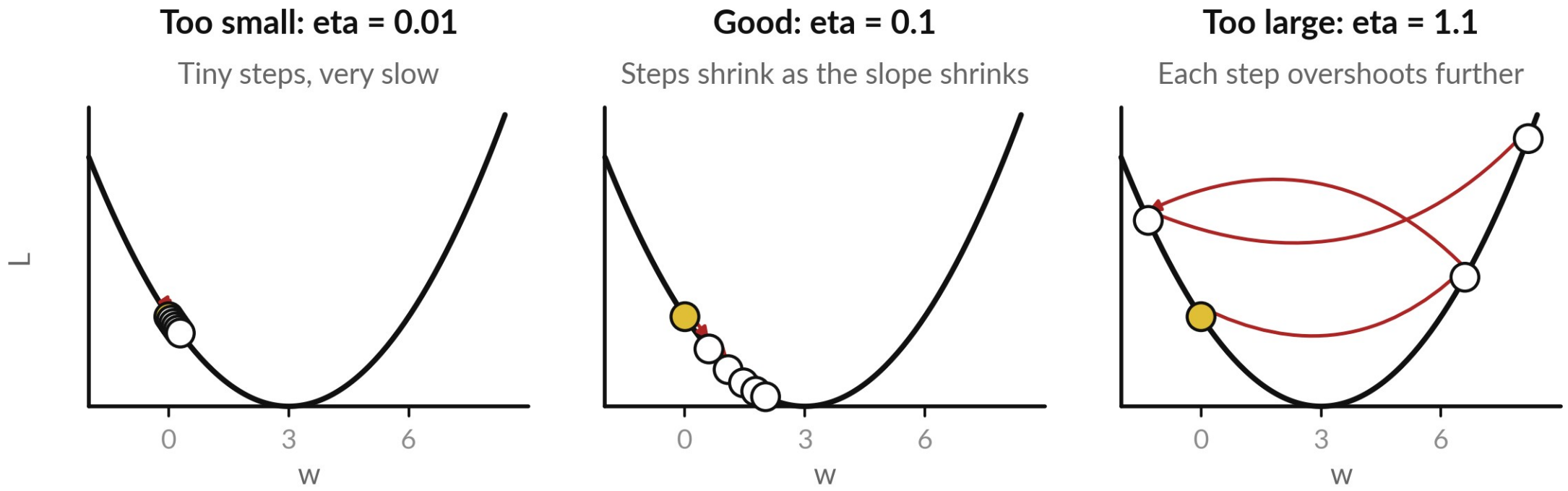
Half sum and SSE

$$L_{half} = \frac{1}{2}\sum_i (\hat{y}_i - y_i)^2 = \frac{n}{2} L_{MSE} \qquad L_{SSE} = \sum_i (\hat{y}_i - y_i)^2 = n\, L_{MSE}$$

Symbol	What it means	In our data
MSE	Loss and slope for w at w = 0, b = 0.	52.11; slope -11.025
Half sum (Session 11)	n/2 = 4 times the MSE, and so is every slope.	208.44; slope -44.10
SSE (Session 12)	n = 8 times the MSE, and so is every slope.	416.88; slope -88.20
Best w and b	The same for all three: multiplying by a positive number moves no minimum.	w = 1.05, b = 6.80

Same step, smaller eta: 0.1 with MSE equals 0.025 with the half sum or 0.0125 with SSE.

The learning rate sets the size of each step: too small crawls, good arrives, too large overshoots



Each panel: L(w) = (w - 3)^2, starting at w = 0. Dots are w after each step; arrows are the steps.

The same three learning rates as numbers: w after each step

Step	eta = 0.01 (too small)	eta = 0.1 (good)	eta = 1.1 (too large)
0	0	0	0
1	0.06	0.6	6.6
2	0.119	1.08	-1.32
3	0.176	1.464	8.184
4	0.233	1.771	-3.221
5	0.288	2.017	10.465
Loss at step 5	7.354	0.966	55.726

The loss at step 0 is 9 in all three columns. With eta = 1.0 exactly, w would bounce between 0 and 6 forever.

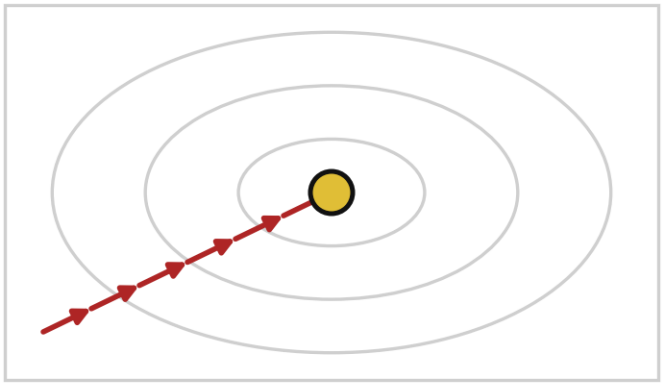
Batch, stochastic and mini-batch gradient descent use different numbers of rows for each step

Batch gradient descent



All 8 rows per step

1 step per epoch

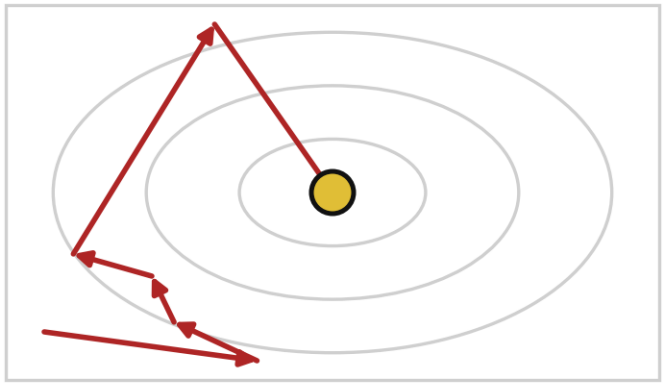


Stochastic (SGD)



1 row per step

8 steps per epoch

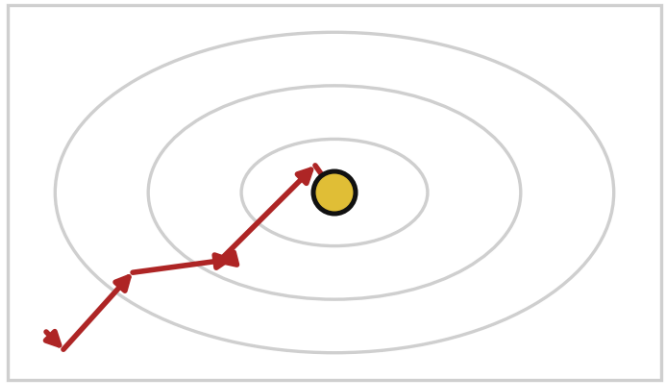


Mini-batch



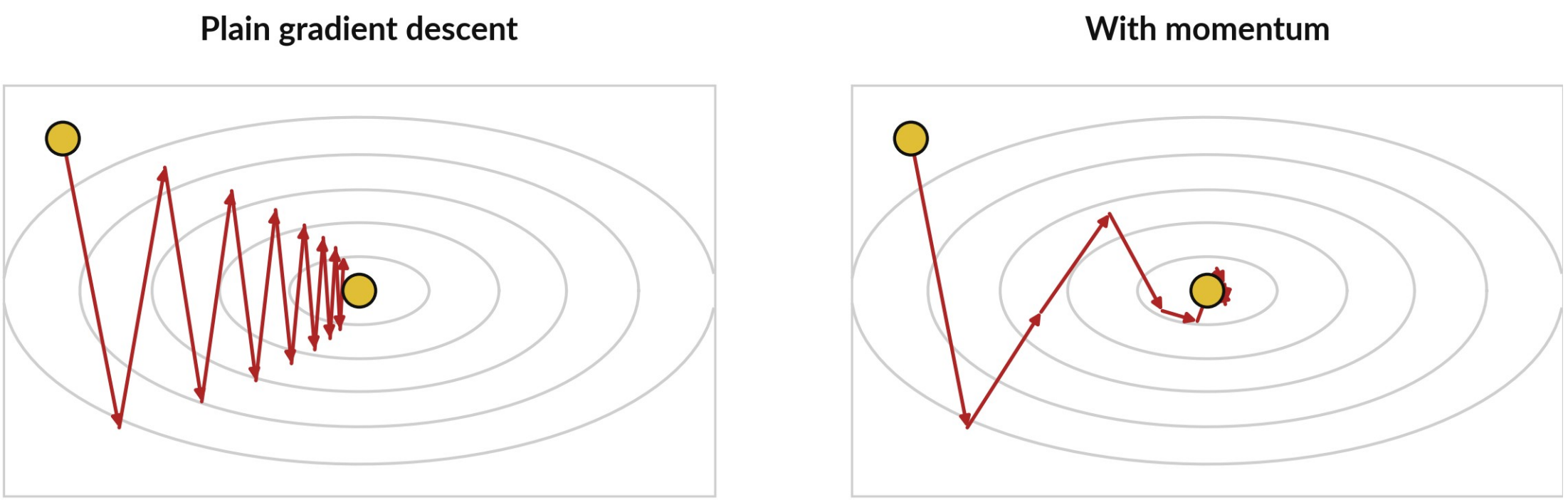
4 rows per step

2 steps per epoch



Gold squares: the rows used for one gradient. An epoch is one pass through all 8 rows.

Momentum keeps a running memory of past gradients, which smooths a zigzag path



Same valley, same start. Momentum damps the sideways swings that make plain descent zigzag.

The momentum velocity is a decaying sum of past gradients

Velocity

$$u_t = \beta\, u_{t-1} + g_t$$

Update

$$w \leftarrow w - \eta\, u_t$$

Symbol	What it means	In our data
g_t	The gradient at step t.	illustrative: g = 2 at every step
u_t	The velocity: a running total of past gradients, each older one counted less. It starts at 0. (Adam's v, which follows, is a different thing.)	u = 2, 3.8, 5.42
beta	The momentum coefficient, between 0 and 1. A common choice is 0.9.	beta = 0.9
eta	The learning rate.	as before

Steady gradient 2: u = 2, then 0.9 × 2 + 2 = 3.8, then 0.9 × 3.8 + 2 = 5.42. It grows towards 20.

Adam divides the average gradient by its typical size, so each parameter steps by about alpha

Average gradient

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t$$

Average squared gradient

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

Update

$$w \leftarrow w - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

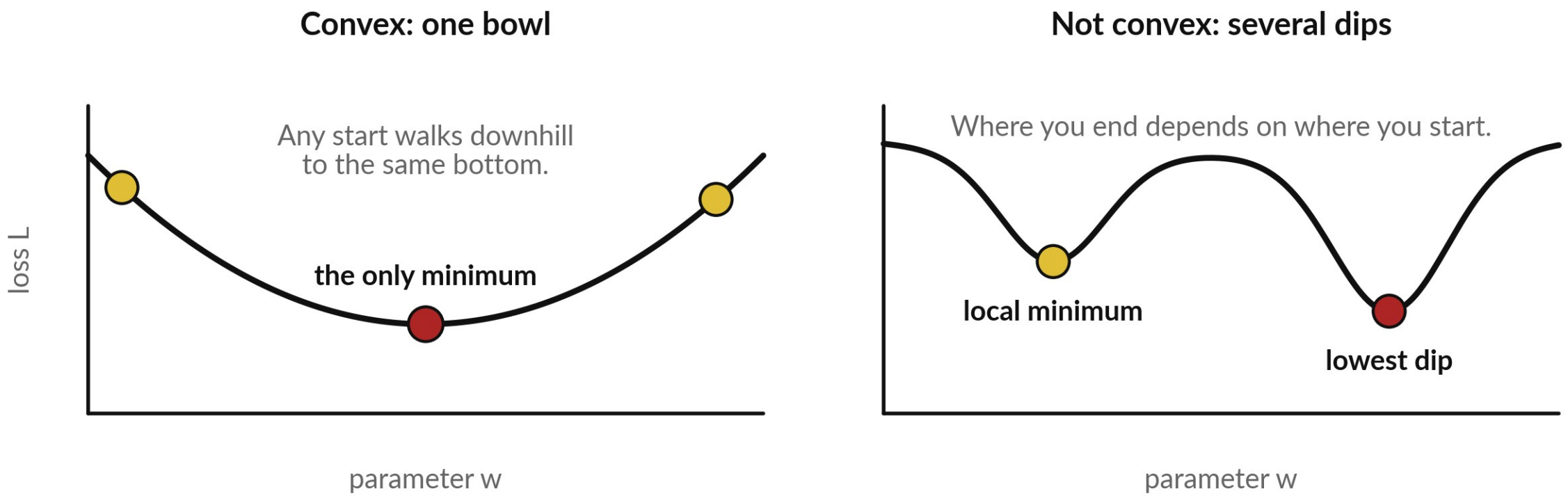
Symbol	What it means	In our data
m_t, v_t	Running averages of the gradients (like momentum) and of the squared gradients. Both start at 0.	beta1 = 0.9, beta2 = 0.999
m-hat, v-hat	The corrected averages. $\beta1^t$ is β1 to the power t, and t is the step number.	$\beta1^t$ = 0.9, 0.81, 0.729 for t = 1, 2, 3
alpha	Adam's learning rate (the paper writes alpha).	0.1 here; the paper suggests 0.001
epsilon	A tiny number that avoids dividing by zero.	1e-8

Adam by hand: three steps on the same bowl, starting at w = 0

t	g	m	v	m-hat	sqrt(v-hat)	Subtracted	w after
1	-6.0000	-0.6000	0.0360	-6.0000	6.0000	-0.1000	0.1000
2	-5.8000	-1.1200	0.0696	-5.8947	5.9008	-0.0999	0.1999
3	-5.6002	-1.5680	0.1009	-5.7861	5.8022	-0.0997	0.2996

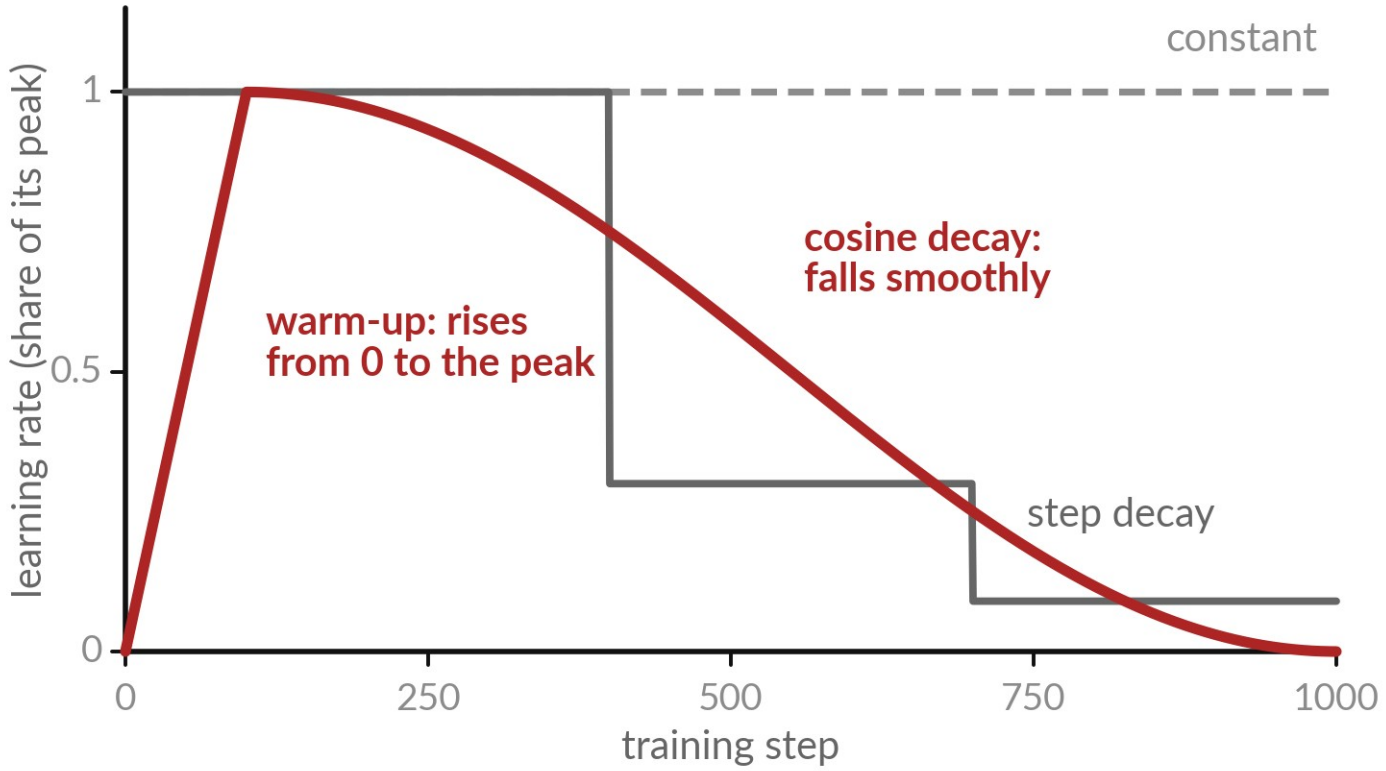
alpha = 0.1, beta1 = 0.9, beta2 = 0.999, epsilon = 1e-8. Subtracted = alpha × m-hat / (sqrt(v-hat) + epsilon). Subtracting a negative amount moves w to the right.

A convex loss has a single bottom, while a network loss can have several dips



Left: a convex loss, one bowl. Right: several dips. A local minimum is a dip that is not the lowest.

A learning-rate schedule changes eta during training



A schedule gives a different learning rate at each step.

Grey: constant, or cut in steps.

Red: warm-up to the peak over 100 steps, then a smooth cosine fall to 0 at step 1000. Peak 0.001: 0.0005 at steps 50 and 550.

Four ways gradient descent goes wrong, and how each one shows itself

Mistake	What you see	Fix
Learning rate too large	The loss goes up, or becomes nan (not a number)	Divide eta by 3 or 10 and run again
Learning rate too small	The loss falls, but so slowly that it looks flat	Multiply eta by 3 or 10, or train for more steps
Wrong sign in the update	The loss climbs from the first step	Use w - eta × gradient, with the minus sign
Inputs on very different scales	One learning rate is too large for one parameter and too small for another	Centre (and, with several inputs, scale) the inputs. The lab centres the distance.

Momentum and Adam both smooth the steps, but Adam also rescales them

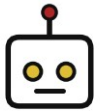


SGD with momentum

One learning rate eta for all parameters

Remembers past gradients

Needs eta tuned with care



Adam

A separate effective step for each parameter

Remembers past gradients and their typical size

Works with the default settings in many cases

Both are optimisers: rules that turn a gradient into a step.

Read the loop before you write it: gradient, then update, repeated

```
for step in range(30):
    err = w * xc + b - y
    dw = 2 * np.mean(err * xc)
    db = 2 * np.mean(err)
    w = w - eta * dw
    b = b - eta * db
```

In plain words

err holds the error y-hat - y of every row.

dw and db are the two slopes of the loss: the gradient.

The last two lines are the update rule.

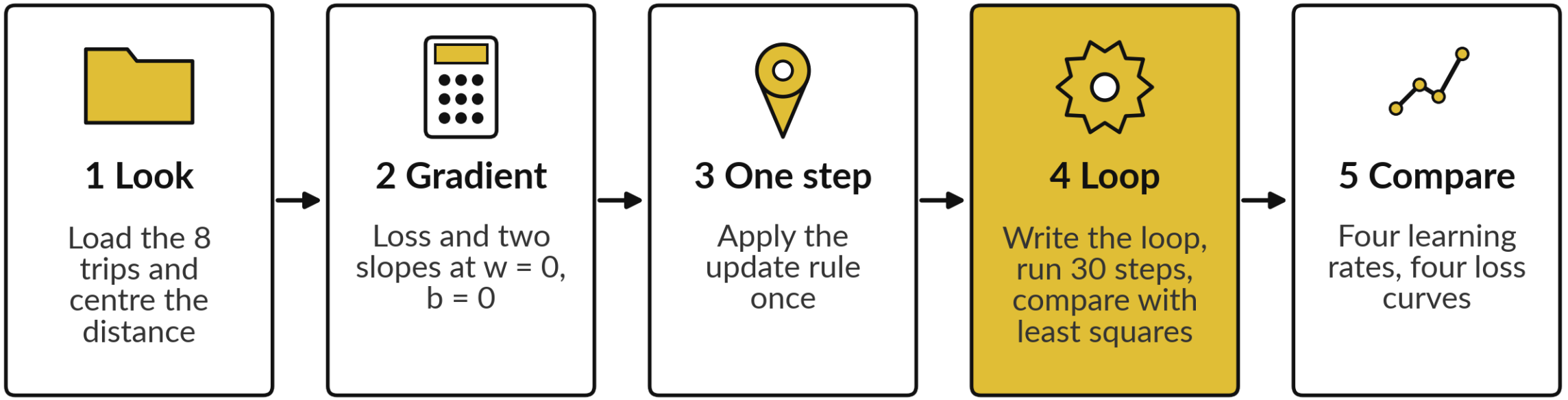
The loop repeats it 30 times. xc is the centred distance.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

Five lab steps take the eight trips from one hand step to four loss curves



The notebook and the workbook follow these five steps. Predict first, run, read the output, then change one thing.

Your lab numbers should match these at every step

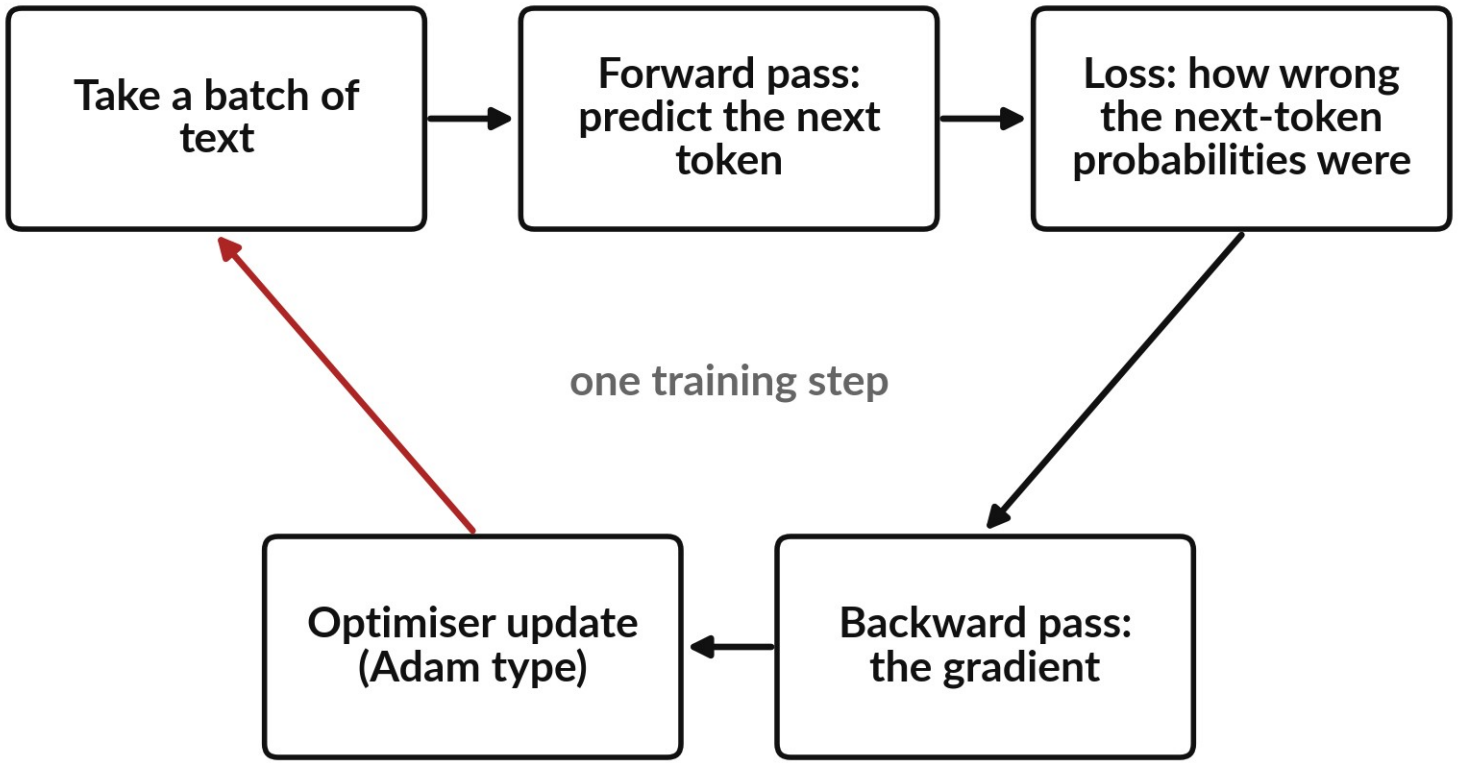
What you compute	You should see
Loss at w = 0, b = 0	52.11
∂L/∂w and ∂L/∂b at w = 0, b = 0	-11.025 and -13.600
w and b after one step with eta = 0.1	1.1025 and 1.3600
Loss after 30 steps, eta = 0.1	0.0819
Best line by the least squares method of Session 4, on these eight trips	w = 1.050, b = 6.800
Loss after 30 steps, eta = 0.01 and 0.05	13.848 and 0.1650
Loss after 30 steps, eta = 0.2	1762.5 (it grew)

If your number differs, find the step that went wrong. Do not change a number to match.

Lab messages, what they mean and what to do next

What you see	What it means	What to do
The loss is nan or inf	The learning rate is too large and the numbers overflowed	Divide eta by 10
NameError in Python	A cell that defines a name was not run	Run all cells from the top
A flat loss curve	The learning rate is too small, or the loop did not update w	Check that w = w - eta * dw is inside the loop
Sheet formula shows #REF! or 0	A row refers to the wrong previous row	Compare with the row above: step t uses row t - 1

Where this appears in AI: every training step ends with an optimiser update



Training is one loop, repeated a very large number of times.

Forward and backward pass: Session 11. Cross-entropy, the loss for next tokens: Session 17.

Published work uses Adam-type optimisers with warm-up. Product settings are not published.

Check an AI assistant with your own numbers

10 minutes

Hands-on, with no private data

Ask any AI assistant to do three Adam steps. Then compare with the table.

Type: "For L(w) = (w - 3)^2, start w = 0, alpha = 0.1, beta1 = 0.9, beta2 = 0.999. Show m, v and w for three Adam steps."

Compare its w after step 3 with the table: 0.2996.

If it differs, find the first number that differs.

Never paste confidential data.

You keep

A note: did the assistant match your table?

Exit check: three questions with short answers

Question	Short answer
With eta = 0.1 and a slope of minus 4, what is the new w if the old w is 1?	w = 1 - 0.1 × (-4) = 1.4.
The loss grows at every step. What is the most likely cause and the fix?	The learning rate is too large. Divide it by 3 or 10.
Give one difference between SGD and Adam.	SGD uses one learning rate for all parameters. Adam rescales the step for each parameter using the typical size of its gradient.

Save your lab and keep the four loss curves

What you keep from today

Keep your saved notebook or workbook, and your four loss curves: learning rates 0.01, 0.05, 0.1 and 0.2. Write one sentence on what each curve shows.

Our question today

How does a model with millions or billions of numbers find good values for all of them?

Next session

Session 11: backpropagation by hand, the gradient of a tiny network with every number written out.

Take away

Update rule

$w = w - eta \times gradient$

Best of the four tried

eta = 0.1, loss 0.082

Too large

eta = 0.2: loss grows

Next session

Backpropagation by hand